

1. CVE 2026-26278: fast-xml-parser affected by DoS through entity expansion in DOCTYPE (no expansion limit)

Package: **fast-xml-parser**

Affected versions

$\geq 4.1.3, < 4.5.4$

$\geq 5.0.0, < 5.3.6$

Patched versions

4.5.4

5.3.6

Description

Summary

The XML parser can be forced to do an unlimited amount of entity expansion. With a very small XML input, it's possible to make the parser spend seconds or even minutes processing a single request, effectively freezing the application.

Details

There is a check in DocTypeReader.js that tries to prevent entity expansion attacks by rejecting entities that reference other entities (it looks for & inside entity values). This does stop classic "Billion Laughs" payloads.

However, it doesn't stop a much simpler variant.

If you define one large entity that contains only raw text (no & characters) and then reference it many times, the parser will happily expand it every time. There is no limit on how large the expanded result can become, or how many replacements are allowed.

The problem is in replaceEntitiesValue() inside OrderedObjParser.js. It repeatedly runs val.replace() in a loop, without any checks on total output size or execution cost. As the entity grows or the number of references increases, parsing time explodes.

Relevant code:

DocTypeReader.js (lines 28–33): entity registration only checks for &

OrderedObjParser.js (lines 439–458): entity replacement loop with no limits

Impact

This is a straightforward denial-of-service issue.

Any service that parses user-supplied XML using the default configuration is vulnerable. Since Node.js runs on a single thread, the moment the parser starts expanding entities, the event loop is blocked. While this is happening, the server can't handle any other requests.

In testing, a payload of only a few kilobytes was enough to make a simple HTTP server completely unresponsive for several minutes, with all other requests timing out.

2. CVE 2020-7660: Serialize JavaScript is Vulnerable to RCE via RegExp.flags and Date.prototype.toISOString()
Package: **serialize-javascript**

Affected versions

<= 7.0.2

Patched versions

7.0.3

Description

Impact

The serialize-javascript npm package (versions <= 7.0.2) contains a code injection vulnerability.

While RegExp.source is sanitized, RegExp.flags is interpolated directly into the generated output without escaping. A similar issue exists in Date.prototype.toISOString().

If an attacker can control the input object passed to serialize(), they can inject malicious JavaScript via the flags property of a RegExp object. When the serialized string is later evaluated (via eval, new Function, or <script> tags), the injected code executes.

3. CVE 2026-25639: Axios is Vulnerable to Denial of Service via __proto__ Key in mergeConfig
Package: **axios**

Affected versions

>= 1.0.0, <= 1.13.4

<= 0.30.2

Patched versions

1.13.5

0.30.3

Description

Denial of Service via proto Key in mergeConfig

Summary

The mergeConfig function in axios crashes with a TypeError when processing configuration objects containing `__proto__` as an own property. An attacker can trigger this by providing a malicious configuration object created via `JSON.parse()`, causing complete denial of service.

Details

The vulnerability exists in `lib/core/mergeConfig.js` at lines 98-101:

```
utils.forEach(Object.keys({ ...config1, ...config2 }), function computeConfigValue(prop) {  
  const merge = mergeMap[prop] || mergeDeepProperties;  
  const configValue = merge(config1[prop], config2[prop], prop);  
  (utils.isUndefined(configValue) && merge !== mergeDirectKeys) || (config[prop] = configValue);  
});
```

When prop is `'__proto__'`:

1. `JSON.parse('{ "__proto__": {...}}')` creates an object with `__proto__` as an own enumerable property
2. `Object.keys()` includes `'__proto__'` in the iteration
3. `mergeMap['__proto__']` performs prototype chain lookup, returning `Object.prototype` (truthy object)
4. The expression `mergeMap[prop] || mergeDeepProperties` evaluates to `Object.prototype`
5. `Object.prototype(...)` throws `TypeError: merge is not a function`

The mergeConfig function is called by:

- `Axios.request()` at `lib/core/Axios.js:75`
- `Axios.getUri()` at `lib/core/Axios.js:201`
- All HTTP method shortcuts (`get`, `post`, etc.) at `lib/core/Axios.js:211,224`

Control tests performed:

Test	Config	Result
Normal config	<code>{"timeout": 5000}</code>	SUCCESS
Malicious config	<code>JSON.parse('{ " __proto__ ": {"x": 1}}')</code>	CRASH

Test	Config	Result
Nested object	{"headers": {"X-Test": "value"}}	SUCCESS

Attack scenario:

An application that accepts user input, parses it with `JSON.parse()`, and passes it to axios configuration will crash when receiving the payload `{"__proto__": {"x": 1}}`.

Impact

Denial of Service - Any application using axios that processes user-controlled JSON and passes it to axios configuration methods is vulnerable. The application will crash when processing the malicious payload.

Affected environments:

- Node.js servers using axios for HTTP requests
- Any backend that passes parsed JSON to axios configuration

This is NOT prototype pollution - the application crashes before any assignment occurs.

4. CVE 2026-25047: deepHas vulnerable to Prototype Pollution via constructor.prototype
Package: **deephas**

Affected versions

< 1.0.8

Patched versions

1.0.8

Description

Summary

A prototype pollution vulnerability exists in version 1.0.7 of the `deephas` npm package that allows an attacker to modify global object behavior. This issue was fixed in version 1.0.8.

Details

The vulnerability resides in the `add()` function and `indexer()` function implemented within `deepHas.js`. Although version 1.0.7 attempts to prevent prototype pollution by checking property ownership (e.g., using `Object.hasOwnProperty`) and by checking against forbidden string usage (using `String.prototype.indexOf`), this check can be bypassed as shown in the PoC

By doing so, an attacker can inject properties into `Object.prototype` through a payload such as `constructor.prototype.polluted` or **proto**.`polluted` resulting in prototype pollution. This issue affects all JavaScript runtimes that rely on npm packages (including Node.js, Deno, and Bun) and is independent of the operating system.

Impact

This is a prototype pollution vulnerability, which can have severe security implications depending on how deephas is used by downstream applications. Any application that processes attacker-controlled input using deephas.set may be affected.

It could potentially lead to the following problems:

- Authentication bypass
- Denial of service
- Remote code execution (if polluted property is passed to sinks like eval or child_process)

5. CVE 2025-59046: interactive-git-checkout has a Command Injection vulnerability

Package: **interactive-git-checkout**

Affected versions

<= 1.1.4

Description

The npm package interactive-git-checkout is an interactive command-line tool that allows users to checkout a git branch while it prompts for the branch name on the command-line. It is available as an npm package and can be installed via `npm install -g interactive-git-checkout`.

Resources:

- Project's npm package: <https://www.npmjs.com/package/interactive-git-checkout>

Command Injection Vulnerability

The interactive-git-checkout tool is vulnerable to a command injection vulnerability because it passes the branch name to the git checkout command using the Node.js child process module's `exec()` function without proper input validation or sanitization.

The following vulnerable code snippets demonstrates the issue:

```
const { exec: execCb } = require('child_process');
const { promisify } = require('util');

const exec = promisify(execCb);

module.exports = async (targetBranch) => {
  const { stdout, stderr } = await exec(`git checkout ${targetBranch}`);
  process.stderr.write(stderr);
  process.stdout.write(stdout);
};
```

Exploit Proof of Concept

1. Install the interactive-git-checkout package (as suggested by the package's README):

```
npm install --global interactive-git-checkout
```

2. Run the executable exposed by the installed package:

```
$ igc
```

3. When prompted, enter the following branch name:

```
hello ; echo 'Command Injection Vulnerability Exploited!' > /tmp/command-injection.txt;  
#
```

6. CVE 2026-25547: @isaacs/brace-expansion has Uncontrolled Resource Consumption

Package: **@isaacs/brace-expansion**

Affected versions

<= 5.0.0

Patched versions

5.0.1

Description

Summary

@isaacs/brace-expansion is vulnerable to a Denial of Service (DoS) issue caused by unbounded brace range expansion. When an attacker provides a pattern containing repeated numeric brace ranges, the library attempts to eagerly generate every possible combination synchronously. Because the expansion grows exponentially, even a small input can consume excessive CPU and memory and may crash the Node.js process.

Details

The vulnerability occurs because @isaacs/brace-expansion expands brace expressions without any upper bound or complexity limit. Expansion is performed eagerly and synchronously, meaning the full result set is generated before returning control to the caller.

For example, the following input:

```
{0..99} {0..99} {0..99} {0..99} {0..99}
```

produces:

$100^5 = 10,000,000,000$ combinations

This exponential growth can quickly overwhelm the event loop and heap memory, resulting in process termination.

Impact

This is a denial of service vulnerability. Any application or downstream dependency that uses @isaacs/brace-expansion on untrusted input may be vulnerable to a single-request crash.

An attacker does not require authentication and can use a very small payload to:

- Trigger exponential computation
- Exhaust memory and CPU resources
- Block the event loop
- Crash Node.js services relying on this library

7. CVE 2026-2631: Command Injection via Unsanitized locate Output in versions() — systeminformation

Package: **systeminformation**

The Vulnerable Code Path

Inside the versions() function, when detecting the PostgreSQL version on Linux, the code does this:

```
// lib/osinfo.js — lines 770-776
exec('locate bin/postgres', (error, stdout) => {
  if (!error) {
    const postgresqlBin = stdout.toString().split('\n').sort();
    if (postgresqlBin.length) {
      exec(postgresqlBin[postgresqlBin.length - 1] + ' -V', (error, stdout) => {
        // parses version string...
      });
    }
  }
});
```

Here's what happens step by step:

1. It runs locate bin/postgres to search the filesystem for PostgreSQL binaries
2. It splits the output by newline and sorts the results alphabetically
3. It takes the **last element** (highest alphabetically)
4. It concatenates that path directly into a new exec() call with + ' -V'

No sanitizeShellString(). No path validation. No execFile(). Raw string concatenation into exec().

The locate command reads from a system-wide database (plocate.db or mlocate.db) that indexes all filenames on the system. If any indexed filename contains shell metacharacters — specifically semicolons — those characters will be interpreted by the shell when passed to exec().

Exploitation

Prerequisites

For this vulnerability to be exploitable, the following conditions must be met:

1. **Target system runs Linux** — the vulnerable code path is inside an if (`_linux`) block
 2. **locate / plocate is installed** — common on Ubuntu, Debian, Fedora, RHEL
 3. **PostgreSQL binary exists in the locate database** — so locate bin/postgres returns results (otherwise the code falls through to a safe `psql -V` fallback)
 4. **The attacker can create files on the filesystem** — in any directory that gets indexed by updatedb
 5. **The locate database gets updated** — updatedb runs daily via systemd timer (plocate-updatedb.timer) or cron on most distros
-
8. CVE2026-29091: locutus `call_user_func_array` vulnerable to Remote Code Execution (RCE) due to Code Injection
Package: **locutus**

Description

Details

A Remote Code Execution (RCE) flaw was discovered in the locutus project (v2.0.39), specifically within the `call_user_func_array` function implementation. The vulnerability allows an attacker to inject arbitrary JavaScript code into the application's runtime environment. This issue stems from an insecure implementation of the `call_user_func_array` function (and its wrapper `call_user_func`), which fails to properly validate all components of a callback array before passing them to `eval()`.

Technical Details

The vulnerability is in the `call_user_func_array` function in `src/php/funchand/call_user_func_array.js`, between lines 31 and 35 of version 2.0.39.

This function mimics PHP's dynamic function call feature and accepts a callback argument, which can be a string (function name) or an array (class and method name). The developers applied a regular expression check (`validJSFunctionNamePattern`) to the first array element (the class identifier), but not to the second element (the method identifier). As a result, the code inserts the user-supplied method name directly into the evaluation string: `func = eval(cb[0] + "[" + cb[1] + "]")`. This oversight allows an attacker to craft a payload in the second element that escapes the property access context, injects arbitrary JavaScript commands, and executes them with the full privileges of the Node.js process.

Impact

If exploited, this issue allows attackers to execute arbitrary JavaScript code in the Node.js process. It occurs when applications pass untrusted array callbacks to `call_user_func_array()`, a practice common in JSON-RPC setups and PHP-to-JavaScript porting layers. Since the library fails to properly sanitize inputs, this is considered a supplier defect rather than an integration error.

This flaw has been exploited in practice, but it is not a "drive-by" vulnerability. It only arises when an application serves as a gateway or router using Locutus functions.

Finally, if an attacker can control `cb[0]` without regex constraints, they could use `global` or `process` directly. However, Locutus protects `cb[0]`. This `cb[1]` injection is the *only* way to bypass the intended security controls of the library. It is a "bypass" of the library's own protection.

9. CVE 2025-14874

Package: **nodemailer**

Description

Summary

A DoS can occur that immediately halts the system due to the use of an unsafe function.

Details

According to **RFC 5322**, nested group structures (a group inside another group) are not allowed. Therefore, in `lib/addressparser/index.js`, the email address parser performs flattening when nested groups appear, since such input is likely to be abnormal. (If the address is valid, it is added as-is.) In other words, the parser flattens all nested groups and inserts them into the final group list.

However, the code implemented for this flattening process can be exploited by malicious input and triggers DoS

RFC 5322 uses a colon (:) to define a group, and commas (,) are used to separate members within a group.

At the following location in lib/addressparser/index.js:

<https://github.com/nodemailer/nodemailer/blob/master/lib/addressparser/index.js#L90>

there is code that performs this flattening. The issue occurs when the email address parser attempts to process the following kind of malicious address header:

g0: g1: g2: g3: ... gN: victim@example.com;

Because no recursion depth limit is enforced, the parser repeatedly invokes itself in the pattern

addressparser → _handleAddress → addressparser → ...

for each nested group. As a result, when an attacker sends a header containing many colons, Nodemailer enters infinite recursion, eventually throwing Maximum call stack size exceeded and causing the process to terminate immediately. Due to the structure of this behavior, no authentication is required, and a single request is enough to shut down the service.

data.group is expected to contain members separated by commas, but in the attacker's payload the group contains colon (:) tokens. Because of this, the parser repeatedly triggers recursive calls for each colon, proportional to their number.

Impact

The attacker can achieve the following:

1. Force an immediate crash of any server/service that uses Nodemailer
2. Kill the backend process with a single web request
3. In environments using PM2/Forever, trigger a continuous restart loop, causing severe resource exhaustion"

10. CVE-2026-24040: jsPDF has Shared State Race Condition in addJS Plugin

Package: jspdf

Description

Impact

The addJS method in the jspdf Node.js build utilizes a shared module-scoped variable (text) to store JavaScript content. When used in a concurrent environment (e.g., a Node.js web server), this variable is shared across all requests.

If multiple requests generate PDFs simultaneously, the JavaScript content intended for one user may be overwritten by a subsequent request before the document is generated. This results in Cross-User Data Leakage, where the PDF generated for User A contains the JavaScript payload (and any embedded sensitive data) intended for User B.

Typically, this only affects server-side environments, although the same race conditions might occur if jsPDF runs client-side.

11. CVE-2026-23745: node-tar is Vulnerable to Arbitrary File Overwrite and Symlink Poisoning via Insufficient Path Sanitization

Package: tar

Description

Summary

The node-tar library ($\leq 7.5.2$) fails to sanitize the linkpath of Link (hardlink) and SymbolicLink entries when preservePaths is false (the default secure behavior). This allows malicious archives to bypass the extraction root restriction, leading to Arbitrary File Overwrite via hardlinks and Symlink Poisoning via absolute symlink targets.

Details

The vulnerability exists in src/unpack.ts within the [HARDLINK] and [SYMLINK] methods.

1. Hardlink Escape (Arbitrary File Overwrite)

The extraction logic uses path.resolve(this.cwd, entry.linkpath) to determine the hardlink target. Standard Node.js behavior dictates that if the second argument (entry.linkpath) is an absolute path, path.resolve ignores the first argument (this.cwd) entirely and returns the absolute path.

The library fails to validate that this resolved target remains within the extraction root. A malicious archive can create a hardlink to a sensitive file on the host (e.g., `/etc/passwd`) and subsequently write to it, if file permissions allow writing to the target file, bypassing path-based security measures that may be in place.

2. Symlink Poisoning

The extraction logic passes the user-supplied `entry.linkpath` directly to `fs.symlink` without validation. This allows the creation of symbolic links pointing to sensitive absolute system paths or traversing paths (`../..`), even when secure extraction defaults are used.

Impact

- **Arbitrary File Overwrite:** An attacker can overwrite any file the extraction process has access to, bypassing path-based security restrictions. It does not grant write access to files that the extraction process does not otherwise have access to, such as root-owned configuration files.
- **Remote Code Execution (RCE):** In CI/CD environments or automated pipelines, overwriting configuration files, scripts, or binaries leads to code execution. (However, npm is unaffected, as it filters out all `Link` and `SymbolicLink` tar entries from extracted packages.)

12. CVE-2026-32141: flatted vulnerable to unbounded recursion DoS in `parse()` `revive` phase

Package: flatted

Description

Summary

flatted's `parse()` function uses a recursive `revive()` phase to resolve circular references in deserialized JSON. When given a crafted payload with deeply nested or self-referential \$ indices, the recursion depth is unbounded, causing a stack overflow that crashes the Node.js process.

Impact

Denial of Service (DoS). Any application that passes untrusted input to `flatted.parse()` can be crashed by an unauthenticated attacker with a single request.

`flatted` has ~87M weekly npm downloads and is used as the circular-JSON serialization layer in many caching and logging libraries.